# Triggers

**2520030575**

**M.LIKHITHA**

## Create a database & use:

```
1   CREATE DATABASE IF NOT EXISTS bookflow2_db;
2 • USE bookflow_db;
```

Output

Action Output

| # | Time | Action |
|---|---|---|
| 1 | 09:08:46 | SHOW DATABASES |
| 2 | 09:09:23 | CREATE DATABASE IF NOT EXISTS bookflow2_db |
| 3 | 09:09:40 | USE bookflow_db |

## Create a table:

```
3 • CREATE TABLE Customer (
4       Customer_ID INT PRIMARY KEY,
5       Customer_Name VARCHAR(100) NOT NULL,
6       Phone VARCHAR(15),
7       Email VARCHAR(100),
8       City VARCHAR(50)
9   );
```

| | | | |
|---|---|---|---|
| 19 | 15:26:30 | USE bookflow9_db | |
| 20 | 15:26:34 | CREATE TABLE Customer ( | Customer_ID INT PRIMARY KEY, Customer_Name VARCHAR(100) N... |

```
10 • CREATE TABLE Account (
11      Account_No INT PRIMARY KEY,
12      Customer_ID INT,
13      Account_Type VARCHAR(20),
14      Balance DECIMAL(12,2) DEFAULT 0,
15      Branch VARCHAR(50),
16
17      FOREIGN KEY (Customer_ID)
18      REFERENCES Customer(Customer_ID)
19  );
```

| # | Time | Action | |
|---|---|---|---|
| 20 | 15:26:34 | CREATE TABLE Customer ( | Customer_ID INT PRIMARY KEY, Customer_Name VARCHAR(100) |
| 21 | 15:27:34 | CREATE TABLE Account ( | Account_No INT PRIMARY KEY, Customer_ID INT, Account_Typ |

```sql
CREATE TABLE Bank_Transaction (
    Transaction_ID INT PRIMARY KEY AUTO_INCREMENT,
    Account_No INT,
    Transaction_Type VARCHAR(20),
    Amount DECIMAL(12,2),
    Transaction_Date DATETIME DEFAULT CURRENT_TIMESTAMP,

    FOREIGN KEY (Account_No)
    REFERENCES Account(Account_No)
);
```

```
Output
Action Output
#  Time      Action
21 15:27:34  CREATE TABLE Account (  Account_No INT PRIMARY KEY,  Customer_ID INT,  Account_Type ...
22 15:28:32  CREATE TABLE Bank_Transaction (  Transaction_ID INT PRIMARY KEY AUTO_INCREMENT,  Ac...
```

```sql
CREATE TABLE Loan (
    Loan_ID INT PRIMARY KEY,
    Customer_ID INT,
    Loan_Type VARCHAR(30),
    Loan_Amount DECIMAL(12,2),
    Interest_Rate DECIMAL(5,2),

    FOREIGN KEY (Customer_ID)
    REFERENCES Customer(Customer_ID)
);
```

```
Output
Action Output
#  Time      Action
22 15:28:32  CREATE TABLE Bank_Transaction (  Transaction_ID INT PRIMARY KEY AUTO_INCREMENT,  Ac
23 15:29:04  CREATE TABLE Loan (  Loan_ID INT PRIMARY KEY,  Customer_ID INT,  Loan_Type VARCHAR
```

## Insert data into customer table:

```sql
INSERT INTO Customer
(Customer_ID, Customer_Name, Phone, Email, City)
VALUES
(101, 'Likhitha', '9876543210', '@gmail.com', 'Hyderabad'),
(102, 'Tejaswini', '9876543211', 'priya@gmail.com', 'Vijayawada'),
(103, 'Pranavi', '9876543212', 'arjun@gmail.com', 'Bangalore'),
(104, 'Mounika', '9876543213', 'sneha@gmail.com', 'Chennai'),
(105, 'Ganesh', '9876543214', 'kiran@gmail.com', 'Hyderabad');
SELECT * FROM Customer;
```

Result Grid | Filter Rows: | Edit: | Export/Import:

| Customer_ID | Customer_Name | Phone | Email | City |
|---|---|---|---|---|
| 101 | Likhitha | 9876543210 | @gmail.com | Hyderabad |
| 102 | Tejaswini | 9876543211 | priya@gmail.com | Vijayawada |
| 103 | Pranavi | 9876543212 | arjun@gmail.com | Bangalore |
| 104 | Mounika | 9876543213 | sneha@gmail.com | Chennai |
| 105 | Ganesh | 9876543214 | kiran@gmail.com | Hyderabad |

## Insert into account:

```sql
INSERT INTO Account
(Account_No, Customer_ID, Account_Type, Balance, Branch)
VALUES
(10001, 101, 'Savings', 50000, 'Hyderabad'),
(10002, 102, 'Savings', 75000, 'Vijayawada'),
(10003, 103, 'Current', 120000, 'Bangalore'),
(10004, 104, 'Savings', 45000, 'Chennai'),
(10005, 105, 'Current', 90000, 'Karnataka');
SELECT * FROM Account;
```

Result Grid | Filter Rows: | Edit: | Export/Import:

| Account_No | Customer_ID | Account_Type | Balance | Branch |
|---|---|---|---|---|
| 10001 | 101 | Savings | 50000.00 | Hyderabad |
| 10002 | 102 | Savings | 75000.00 | Vijayawada |
| 10003 | 103 | Current | 120000.00 | Bangalore |
| 10004 | 104 | Savings | 45000.00 | Chennai |
| 10005 | 105 | Current | 90000.00 | Karnataka |

## Insert into bank transaction:

```
INSERT INTO Bank_Transaction
(Account_No, Transaction_Type, Amount)
VALUES
(10001, 'DEPOSIT', 10000),
(10002, 'DEPOSIT', 15000),
(10003, 'WITHDRAW', 20000),
(10004, 'DEPOSIT', 5000),
(10005, 'WITHDRAW', 10000);
SELECT * FROM Bank_Transaction;
```

Result Grid | Filter Rows: | Edit: | Export/Import

| Transaction_ID | Account_No | Transaction_Type | Amount | Transaction_Date |
|---|---|---|---|---|
| 1 | 10001 | DEPOSIT | 10000.00 | 2026-08-16 15:33:53 |
| 2 | 10002 | DEPOSIT | 15000.00 | 2026-08-16 15:33:53 |
| 3 | 10003 | WITHDRAW | 20000.00 | 2026-08-16 15:33:53 |
| 4 | 10004 | DEPOSIT | 5000.00 | 2026-08-16 15:33:53 |
| 5 | 10005 | WITHDRAW | 10000.00 | 2026-08-16 15:33:53 |

## Insert into loan:

```
INSERT INTO Loan
(Loan_ID, Customer_ID, Loan_Type, Loan_Amount, Interest_Rate)
VALUES
(501, 101, 'Home Loan', 5000000, 7.5),
(502, 102, 'Education Loan', 1000000, 6.5),
(503, 103, 'Car Loan', 800000, 8.2),
(504, 104, 'Personal Loan', 500000, 10.5);
SELECT * FROM Loan
```

Result Grid | Filter Rows: | Edit: | Export/Import:

| Loan_ID | Customer_ID | Loan_Type | Loan_Amount | Interest_Rate |
|---|---|---|---|---|
| 501 | 101 | Home Loan | 5000000.00 | 7.50 |
| 502 | 102 | Education Loan | 1000000.00 | 6.50 |
| 503 | 103 | Car Loan | 800000.00 | 8.20 |
| 504 | 104 | Personal Loan | 500000.00 | 10.50 |

## Procedure to display all customers:

```
DELIMITER //
CREATE PROCEDURE GetAllCustomers()
BEGIN
    SELECT * FROM Customer;
END //
DELIMITER ;
CALL GetAllCustomers();
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

| Customer_ID | Customer_Name | Phone | Email | City |
|---|---|---|---|---|
| 101 | Likhitha | 9876543210 | @gmail.com | Hyderabad |
| 102 | Tejaswini | 9876543211 | priya@gmail.com | Vijayawada |
| 103 | Pranavi | 9876543212 | arjun@gmail.com | Bangalore |
| 104 | Mounika | 9876543213 | sneha@gmail.com | Chennai |
| 105 | Ganesh | 9876543214 | kiran@gmail.com | Hyderabad |

## Procedure to get account details:

```
CREATE PROCEDURE GetAccountDetails(
    IN p_Account_No INT
)
BEGIN
    SELECT *
    FROM Account
    WHERE Account_No = p_Account_No;
END //
DELIMITER ;
CALL GetAccountDetails(10001);
```

Result Grid | Filter Rows: | Export: | Wrap Cell Co

| Account_No | Customer_ID | Account_Type | Balance | Branch |
|---|---|---|---|---|
| 10001 | 101 | Savings | 50000.00 | Hyderabad |

## Procedure to find customer accounts:

```
BEGIN
    SELECT
        C.Customer_ID,
        C.Customer_Name,
        A.Account_No,
        A.Account_Type,
        A.Balance,
        A.Branch
    FROM Customer C
    JOIN Account A
    ON C.Customer_ID = A.Customer_ID
    WHERE C.Customer_ID = p_Customer_ID;
    END //
DELIMITER ;
CALL GetCustomerAccounts(101);
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

| Customer_ID | Customer_Name | Account_No | Account_Type | Balance | Branch |
|---|---|---|---|---|---|
| 101 | Likhitha | 10001 | Savings | 50000.00 | Hyderabad |

Procedure to deposit money:

```
DELIMITER //
CREATE PROCEDURE DepositMoney(
    IN p_Account_No INT,
    IN p_Amount DECIMAL(12,2)
)
BEGIN
    UPDATE Account
    SET Balance = Balance + p_Amount
    WHERE Account_No = p_Account_No;
END //
DELIMITER ;
CALL DepositMoney(10001, 5000);
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

| Customer_ID | Customer_Name | Account_No | Account_Type | Balance | Branch |
|---|---|---|---|---|---|
| 101 | Likhitha | 10001 | Savings | 60000.00 | Hyderabad |

Procedure to withdraw money:

```
DELIMITER //
CREATE PROCEDURE WithdrawMoney(
    IN p_Account_No INT,
    IN p_Amount DECIMAL(12,2)
)
BEGIN
    UPDATE Account
    SET Balance = Balance - p_Amount
    WHERE Account_No = p_Account_No;
END //
DELIMITER ;
CALL WithdrawMoney(10001, 3000);
```

Trigger- Prevent negative balance:

```
DELIMITER //
CREATE TRIGGER CheckBalance
BEFORE UPDATE ON Account
FOR EACH ROW
BEGIN
    IF NEW.Balance < 0 THEN
        SIGNAL SQLSTATE '45000'
        SET MESSAGE_TEXT =
        'Transaction failed: Insufficient balance';
    END IF;
END //
DELIMITER ;
UPDATE Account
SET Balance = Balance - 60000
WHERE Account_No = 10001;
```

Trigger- Prevent Negative deposit:

```
DELIMITER //
CREATE TRIGGER CheckTransactionAmount
BEFORE INSERT ON Bank_Transaction
FOR EACH ROW
BEGIN
    IF NEW.Amount <= 0 THEN
        SIGNAL SQLSTATE '45000'
        SET MESSAGE_TEXT =
        'Transaction amount must be greater than zero';
    END IF;
END //
DELIMITER ;
INSERT INTO Bank_Transaction
(Account_No, Transaction_Type, Amount)
VALUES
(10001, 'DEPOSIT', -5000);
```

Create Transaction audit table:

```
CREATE TABLE Transaction_Audit (
    Audit_ID INT PRIMARY KEY AUTO_INCREMENT,
    Transaction_ID INT,
    Account_No INT,
    Transaction_Type VARCHAR(20),
    Amount DECIMAL(12,2),
    Audit_Date DATETIME DEFAULT CURRENT_TIMESTAMP
);
```

```
40  13:17:11  CREATE TABLE Transaction_Audit (    Audit_ID INT PRIMARY KEY AUTO_INCREMENT,
```

Trigger – transaction audit:

```
DELIMITER //
CREATE TRIGGER TransactionAudit
AFTER INSERT ON Bank_Transaction
FOR EACH ROW
BEGIN
    INSERT INTO Transaction_Audit
    (
        Transaction_ID,
        Account_No,
        Transaction_Type,
        Amount
    )
    VALUES
    (
        NEW.Transaction_ID,
        NEW.Account_No,
        NEW.Transaction_Type,
        NEW.Amount
    );
END //
DELIMITER ;
INSERT INTO Bank_Transaction
(Account_No, Transaction_Type, Amount)
VALUES
(10001, 'DEPOSIT', 1500);
SELECT * FROM Transaction_Audit;
```

| | Audit_ID | Transaction_ID | Account_No | Transaction_Type | Amount | Audit_Date |
|---|---|---|---|---|---|---|
| * | NULL | NULL | NULL | NULL | NULL | NULL |

Procedure – Account Transfer

```
DELIMITER //
CREATE TRIGGER PreventInsufficientWithdrawal
BEFORE INSERT ON Bank_Transaction
FOR EACH ROW
BEGIN
    DECLARE CurrentBalance DECIMAL(12,2);
    SELECT Balance
    INTO CurrentBalance
    FROM Account
    WHERE Account_No = NEW.Account_No;
    IF NEW.Transaction_Type = 'WITHDRAW'
       AND NEW.Amount > CurrentBalance THEN
        SIGNAL SQLSTATE '45000'
        SET MESSAGE_TEXT =
        'Withdrawal failed: Insufficient balance';
    END IF;
END //
DELIMITER ;
INSERT INTO Bank_Transaction
(Account_No, Transaction_Type, Amount)
VALUES
(10001, 'WITHDRAW', 1000000);
```